

DU CLIENTS-SERVEUR AU PEER TO PEER AVEC DES TUBES

Notre objectif est de simuler un système de téléchargement en Peer to Peer. Le système sera composé de 10 nœuds (maximum) nommés 0, 1, 2, ...9. Le nœud 0 contiendra un message composé de 10 lettres (par exemple la chaîne `excalibur!`). L'objectif de tous les autres nœuds est de télécharger le message.

- Pour communiquer entre eux, les nœuds auront chacun un tube nommé `tube<num>` qui leur servira de boîte aux lettres. Un nœud pourra donc lire son tube et aussi écrire dans tous les autres tubes. (chaque nœud sait qu'il y a 10 nœuds en tout)
- Les messages envoyés sont de 2 types :
 - **Demande de lettre** : ce type de message commence par la lettre D puis le numéro de l'émetteur puis le numéro de la lettre demandée. Par exemple `D13` est une demande du nœud 1 pour avoir la lettre numéro 3
 - **Envoi de lettre** : ce type de message commence par la lettre E puis le numéro de la lettre demandée, puis la valeur de la lettre demandée. Par exemple `E3a` est l'envoi de la lettre numéro 3 qui vaut `a`. Si le nœud qui reçoit la demande ne possède pas la lettre, il renverra le caractère `-` pour signifier qu'il n'a pas la lettre. Afin de simuler le temps de téléchargement, lorsque le serveur a la lettre demandée, il attendra 1s avant d'envoyer la lettre. Si en revanche il n'a pas la lettre il répondra immédiatement.

L'objectif du TP est de trouver un mode d'organisation qui permette de minimiser le temps total nécessaire pour que tous les clients aient le message complet. (dans le mode client serveur traditionnel, il faudra $10 \times 10 = 100$ s pour atteindre cet objectif)

Nous allons procéder par étapes successives.

Étape 1 : un serveur

- Nous allons d'abord créer un serveur (nœud 0) qui possède le message `excalibur!` dans sa mémoire et devra écouter sur son tube (`tube0`) les demandes de lettre de la part d'un client.

Pour tester le serveur, nous utiliserons des commandes `bash` :

- dans un terminal créer un tube nommé `tube1`, où le serveur pourra envoyer sa réponse
- se mettre en attente de la réponse sur le `tube1` grâce à la commande : `tail -f tube1`
- dans un autre terminal envoyer le message au serveur on pourra utiliser la commande :
`echo -n D13 > tube0`
- vérifier alors que la réponse `E3a` a bien été reçue dans le premier terminal

Étape 2 : des clients patients

- Créer maintenant un client (dont le numéro est passé en paramètre), qui va mettre en place l'algorithme suivant :
 1. Il envoie une demande au serveur (noeud0) pour récupérer la première lettre (lettre 0)
 2. Il attend la réponse de sa première lettre avant de demander la suivante
 3. Et ainsi de suite jusqu'à obtenir les 10 lettres du message
 4. A la fin il affiche le message et le temps total mis pour l'obtenir
- Lancer plusieurs clients en même temps et regarder quel est le temps maximum pour l'obtention de tous les messages de tous les clients (vous devez constater qu'avec n clients, le téléchargement du message va n fois moins vite)

Étape 3 : un client devient serveur

On va s'approcher du fonctionnement Peer to Peer en permettant à un client de devenir serveur et donc on même temps qu'il demandera des lettres au serveur il pourra aussi répondre aux demandes de lettres d'autres nœuds.

- Modifier donc l'algorithme du client de façon à ce que pendant l'attente de la réponse d'une lettre au serveur il traite les messages de demande de lettres qui lui sont adressées et qu'il y réponde. En cas de demande de lettre, si il possède la lettre il attend 1s et répond, si il ne la possède pas, il répond immédiatement.
- Tester en lançant le client-serveur dans une console.
- Et dans une autre console effectuer des tests similaires à l'étape1

Étape 4 : Peer to Peer version1

Afin de finir de mettre en œuvre le Peer to Peer :

- modifier l'algorithme du client-serveur afin que au lieu d'envoyer une demande de lettre au serveur, il puisse envoyer la demander aléatoirement au serveur ou à chacun des clients (on suppose qu'il y a au plus 9 clients)
- Constatez alors en lançant plusieurs clients-serveurs le temps mis pour télécharger le message chez tout le monde
- Pour améliorer les performances, vous pouvez demander les lettres dans un ordre aléatoire plutôt que de 0 à 9